

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОРДЕНА ТРУДОВОГО КРАСНОГО ЗНАМЕНИ ФЕДЕРАЛЬНОЕ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ»

Отчет по учебной практике

«Навигатор по смыслу» — сравнение моделей семантического поиска

Выполнил студент группы БВТ2502

Вершинина Кристина Денисовна

Москва 2026

Введение

В современных системах поиска информации по исходному коду традиционные методы, основанные на точном совпадении текста, демонстрируют низкую эффективность.

Характерным примером является ситуация, когда разработчик вводит запрос «обработка ошибок», а в коде искомая функция названа `exception_handler`, и поиск не находит нужный фрагмент.

Решением данной проблемы выступает семантический поиск на основе эмбедингов — числовых векторов, кодирующих смысл текста.

При использовании эмбедингов семантически похожие фрагменты кода оказываются близко расположенными в многомерном математическом пространстве.

Целью настоящей работы является сравнение двух моделей эмбедингов на небольшом датасете и определение того, какая модель работает лучше и почему.

Весь код разрабатывается исключительно в среде Jupyter Notebook, использование сторонних сервисов, фронтенда и Docker не допускается.

Постановка задачи

Исследование включало три этапа:

1) загрузка датасета из 100 фрагментов кода и выбор двух моделей эмбедингов (одна рекомендована организаторами (`paraphrase-multilingual-MiniLM-L12-v2` и `paraphrase-multilingual-mpnet-base-v2`), вторая выбрана самостоятельно);

2) генерация эмбедингов для всех фрагментов и реализация поиска топ-3 релевантных ответов для 25 тестовых вопросов на основе косинусного сходства;

3) расчет метрики `Precision@3`, построение сводной таблицы и визуализация `t-SNE` для лучшей модели. Дополнительно проведены анализ ошибок по категориям и сравнение качества на русских и английских

запросах.

Ход выполнения работы

На начальном этапе работы производится установка всех необходимых библиотек, включая sentence-transformers, numpy, pandas, matplotlib и scikit-learn.

Установка выполняется с помощью команды `pip install -r requirements.txt`, что гарантирует совместимость версий и воспроизводимость результатов.

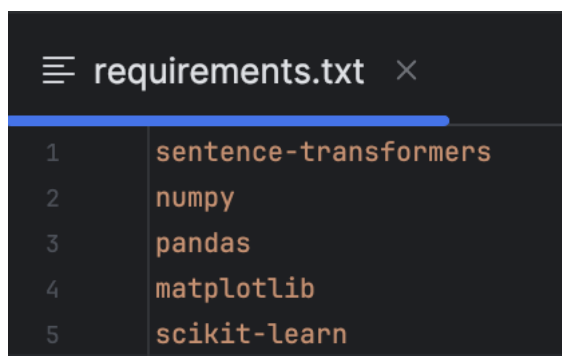


Рис.1 – содержание файла requirements.txt

```
(venv) vonucienogi@MacBook-Air-vonucie practice % pip install -r requirements.txt
```

Рис.2 – установка необходимых библиотек.

```
(venv) vonucienogi@MacBook-Air-vonucie practice % jupyter notebook
```

Рис.3 – запуск Jupyter Notebook.

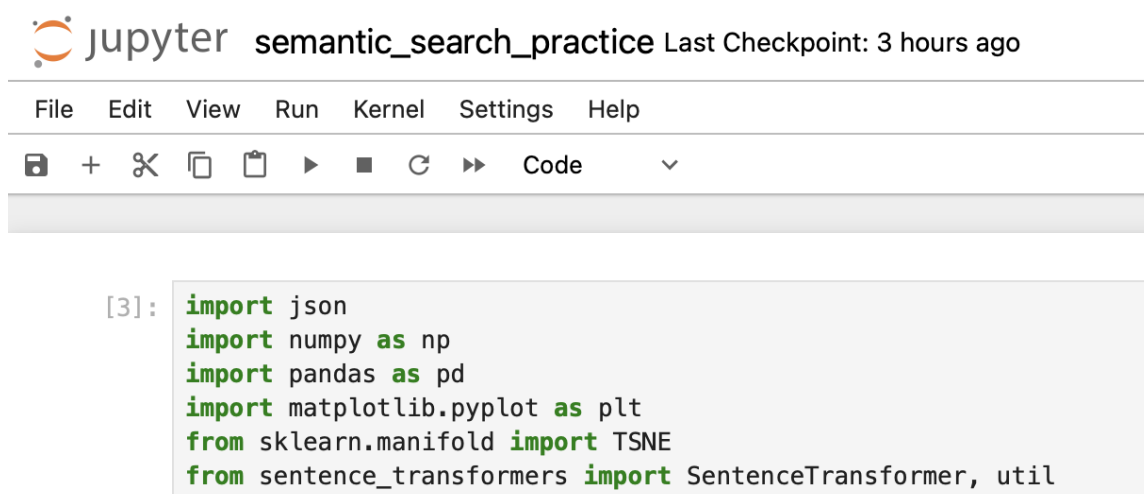


Рис.4 – установка необходимых библиотек в Jupyter Notebook (json для чтения json-файлов с данными, numpy для математических операций, pandas для таблицы, matplotlib.pyplot для графика t-SNE, sklearn.manifold.TSNE для

двумерной проекции, `sentence_transformers` для загрузки моделей и эмбеддингов).

```
[5]: with open("code_corpus.json", "r", encoding="utf-8") as f:
      code_corpus = json.load(f)

      with open("eval_questions.json", "r", encoding="utf-8") as f:
          eval_questions = json.load(f)

      with open("categories.json", "r", encoding="utf-8") as f:
          categories_data = json.load(f)

      print(f"количество функций: {len(code_corpus)}")
      print(f"количество вопросов: {len(eval_questions)}")
      print(f"количество категорий: {len(categories_data['categories'])}")

количество функций: 200
количество вопросов: 25
количество категорий: 5
```

Рис.5 – загрузка данных из json-файлов.

Затем в Jupyter Notebook загружаются выбранные модели эмбеддингов с использованием класса `SentenceTransformer` и указанием имени модели.

Были выбраны следующие две модели эмбеддингов:

1. `paraphrase-multilingual-mpnet-base-v2` (рекомендованная организаторами)

Модель обеспечивает высокое качество векторизации текстов на разных языках, включая русский, при умеренных вычислительных затратах. Размерность эмбеддинга — 768, что позволяет эффективно хранить и обрабатывать векторы. Является одной из наиболее сбалансированных моделей в библиотеке `sentence-transformers`.

2. `LaBSE` (самостоятельный выбор)

`LaBSE` (Language-agnostic BERT Sentence Embedding) — модель от Google, созданная специально для мультязычных задач. Размер эмбеддинга — 768. Модель позволяет находить смысловые соответствия между текстами на разных языках, что важно для работы с русскими и английскими запросами.

В отличие от `MPNet`, `LaBSE` целенаправленно обучалась на параллельных текстах для кросс-языкового выравнивания, что делает её потенциально более эффективной для запросов на русском языке. Несмотря на одинаковую

размерность эмбедингов, эти модели по-разному распределяют тексты в векторном пространстве. Например, при кластеризации текстов LaBSE создаёт более чёткие и стабильные тематические кластеры, чем MPNet.

```
[9]: model_mpnet = SentenceTransformer("paraphrase-multilingual-mpnet-base-v2")
     model_labse = SentenceTransformer("LaBSE")
     print("модели загружены")
```



модели загружены

Рис.6 – установка выбранных моделей эмбедингов.

Далее производится чтение файла `code_corpus.json` и извлечение из него всех 100 фрагментов кода, которые преобразуются в эмбединги.

Полученные векторы сохраняются в отдельные файлы, например в формате NumPy, для последующего использования на этапе поиска.

После этого загружается файл `eval_questions.json`, содержащий 25 тестовых вопросов.

```
[23]: corpus_texts = []
     corpus_ids = []
     corpus_categories = []

     for item in code_corpus:
         corpus_texts.append(f"{item['function_name']}: {item['description']}")
         corpus_ids.append(item['id'])
         corpus_categories.append(item['category'])

     print(f"подготовлено {len(corpus_texts)} текстов")
```

подготовлено 200 текстов

Рис.7 – подготовка текстов.

Для каждого вопроса вычисляется его эмбединг, после чего рассчитывается косинусное сходство между вектором вопроса и векторами всех 100 фрагментов кода.

Косинусное сходство может быть вычислено как с использованием стандартной формулы через numpy, так и с применением встроенной функции `util.cos_sim` из библиотеки `sentence-transformers`.

На основе полученных значений сходства выполняется сортировка фрагментов по убыванию и выбираются первые три, которые считаются

наиболее релевантными ответами на заданный вопрос.

```
[30]: embeddings_mpnet = model_mpnet.encode(corpus_texts, show_progress_bar=True)
      np.save("embeddings_mpnet.npy", embeddings_mpnet)
      print("MPNet готово")
```

Batches: 100%  7/7 [00:01<00:00, 7.12it/s]
MPNet готово

```
[31]: embeddings_labse = model_labse.encode(corpus_texts, show_progress_bar=True)
      np.save("embeddings_labse.npy", embeddings_labse)
      print("LaBSE готово")
```

Batches: 100%  7/7 [00:05<00:00, 2.48it/s]
LaBSE готово

Рис.8 – генерация эмбедингов с использованием модели MPNet и модели LaBSE.

```
[36]: def search(query, model, embeddings):
      q_emb = model.encode(query)
      scores = np.dot(embeddings, q_emb) / (np.linalg.norm(embeddings, axis=1) * np.linalg.norm(q_emb))
      top_idx = np.argsort(scores)[-3:][::-1]
      results = []
      for idx in top_idx:
          results.append({
              'id': corpus_ids[idx],
              'score': float(scores[idx])
          })
      return results
```

Рис.9 – функция поиска релевантных фрагментов.

Затем для каждой модели определяется, для скольких вопросов правильный ответ, указанный в поле `correct_chunk_id`, попал в число трёх выбранных фрагментов.

На основе этих данных вычисляется значение Precision@3 как отношение числа успешно отвеченных вопросов к общему числу вопросов.

```
[37]: def precision_at_3(model, embeddings):
      correct = 0
      for q in eval_questions:
          top = search(q["query"], model, embeddings)
          top_ids = [r["id"] for r in top]
          if q["correct_chunk_id"] in top_ids:
              correct += 1
      return correct / len(eval_questions)

p_mpnet = precision_at_3(model_mpnet, embeddings_mpnet)
p_labse = precision_at_3(model_labse, embeddings_labse)

print(f"MPNet Precision@3: {p_mpnet:.3f}")
print(f"LaBSE Precision@3: {p_labse:.3f}")
```

MPNet Precision@3: 0.920
LaBSE Precision@3: 0.960

Рис.10 – функция значение Precision@3 как отношение числа успешно отвеченных вопросов к общему числу вопросов.

После расчёта метрики для всех моделей формируется сводная таблица, в которой для каждой модели указывается соответствующее значение Precision@3.

```
[40]: df = pd.DataFrame([
    ["paraphrase-multilingual-mpnet-base-v2", p_mpnet],
    ["LaBSE", p_labse]
], columns=["модель", "Precision@3"])
df = df.sort_values("Precision@3", ascending=False)
print(df.to_string(index=False))
df.to_csv("model_comparison.csv", index=False)
```

модель	Precision@3
LaBSE	0.96
paraphrase-multilingual-mpnet-base-v2	0.92

Рис.11 – сводная таблица для моделей.

Сводная таблица показала, что модель LaBSE лучше справляется с ответами на вопросы.

Для визуализации качества работы лучшей модели выполняется двумерная проекция эмбедингов всех 100 фрагментов с использованием алгоритма t-SNE из библиотеки scikit-learn.

На полученном графике каждая точка окрашивается в соответствии с тематической категорией фрагмента, что позволяет визуально оценить, насколько хорошо модель разделяет различные тематики в пространстве эмбедингов.

```
[39]: best_emb = embeddings_mpnet if p_mpnet > p_labse else embeddings_labse
best_name = "MPNet" if p_mpnet > p_labse else "LaBSE"

tsne = TSNE(n_components=2, random_state=42)
coords = tsne.fit_transform(best_emb)

colors = {cat["key"]: cat["color"] for cat in categories_data["categories"]}

plt.figure(figsize=(12, 8))
for cat in set(corpus_categories):
    mask = [c == cat for c in corpus_categories]
    plt.scatter(
        coords[mask, 0],
        coords[mask, 1],
        label=cat,
        color=colors[cat],
        alpha=0.7
    )

plt.legend()
plt.title(f"t-SNE: {best_name}")
plt.savefig("tsne_plot.png", dpi=300)
plt.show()
```

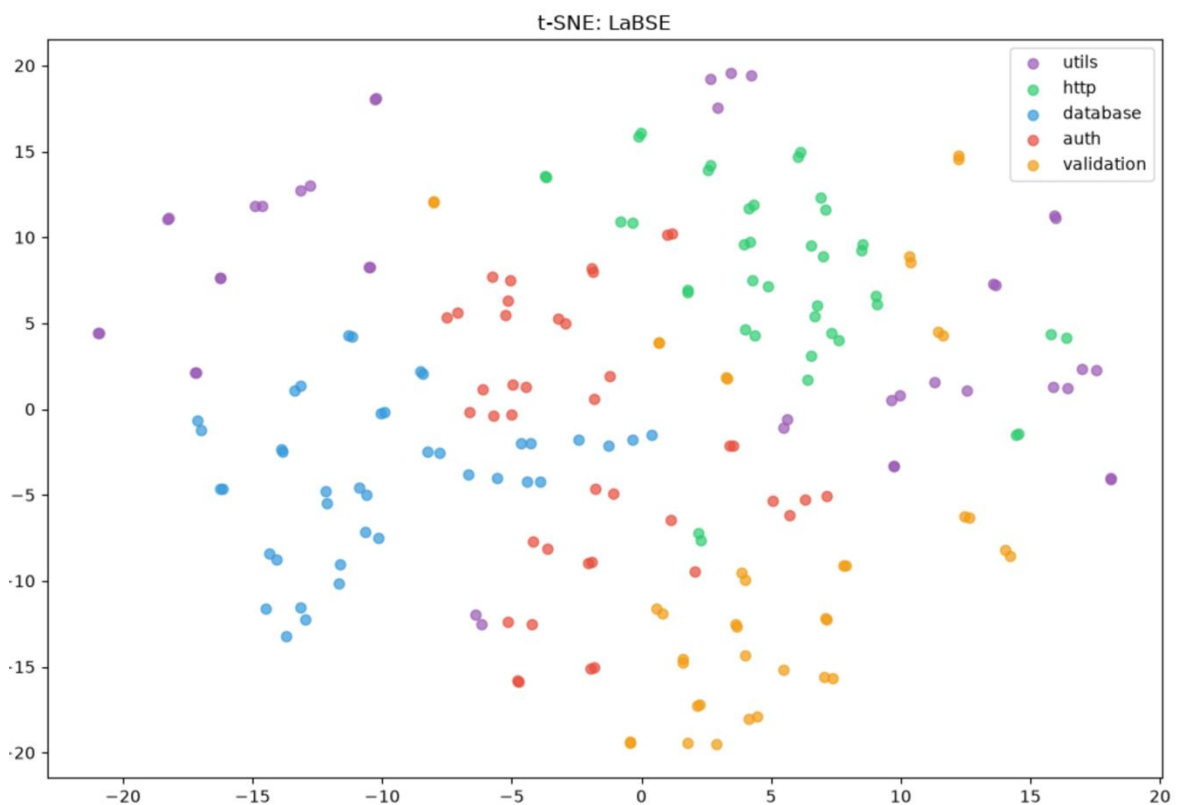


Рис.12 – двумерная проекция эмбедингов всех фрагментов модели LaBSE.

Построенная с помощью t-SNE двумерная проекция эмбедингов подтверждает, что LaBSE формирует четкие тематические кластеры, соответствующие категориям кода (auth, database, http, validation, utils).

При выполнении дополнительных заданий дополнительно анализируются ошибки модели: определяется, для каких тематических категорий вопросов модель чаще всего даёт неверные ответы, и делаются выводы о возможных причинах таких ошибок.

```
[58]: errors = {}
total = {}
for q in eval_questions:
    cat = next(item["category"] for item in code_corpus if item["id"] == q["correct_chunk_id"])
    total[cat] = total.get(cat, 0) + 1
    top = search(q["query"], model_labse, embeddings_labse)
    if q["correct_chunk_id"] not in [r["id"] for r in top]:
        errors[cat] = errors.get(cat, 0) + 1

df_err = pd.DataFrame([{"категория": k, "ошибок": errors.get(k,0), "всего": total[k]} for k in total])
df_err["точность"] = round((df_err["всего"] - df_err["ошибок"]) / df_err["всего"] * 100, 1)
print("модель: LaBSE")
print(df_err.to_string(index=False))
df_err.to_csv("error_analysis.csv", index=False)
```

модель: LaBSE	категория	ошибок	всего	точность
	auth	1	5	80.0
	database	0	5	100.0
	http	0	5	100.0
	validation	0	5	100.0
	utils	0	5	100.0

Рис.13 – таблица ошибок модели по категориям.

Анализ ошибок показал, что модель LaBSE допустила только одну ошибку из 25 вопросов — в категории auth (точность 80%). Это может быть связано с тем, что вопросы по аутентификации часто содержат схожую терминологию, и модель может путать функции, выполняющие разные, но семантически близкие задачи.

Также при наличии дополнительного задания выполняется разделение вопросов на русскоязычные и англоязычные, и для каждой группы вычисляется Precision@3 отдельно, что позволяет оценить влияние языка запроса на качество поиска.

```
[61]: ru = [q for q in eval_questions if q["language"] == "ru"]
en = [q for q in eval_questions if q["language"] == "en"]

def p3(qs):
    if not qs: return 0
    return sum(1 for q in qs if q["correct_chunk_id"] in [r["id"] for r in search(q["query"], model_labse, embeddings_labse)]) / len(qs)

lang_df = pd.DataFrame([
    ["Русский", len(ru), round(p3(ru), 3)],
    ["Английский", len(en), round(p3(en), 3)]
], columns=["Язык", "Вопросов", "Precision@3"])
print("Модель: LaBSE")
print(lang_df.to_string(index=False))
lang_df.to_csv("language_comparison.csv", index=False)
```

Модель: LaBSE	Язык	Вопросов	Precision@3
	Русский	15	0.933
	Английский	10	1.000

Рис.14 – вычисление функции Precision@3 по категориям на русском и

английском языке.

Сравнение качества на русских и английских запросах показало, что модель LaBSE достигла Precision@3 0.933 на русском языке и 1.000 на английском. Небольшое преимущество англоязычных запросов может объясняться тем, что исходные тексты функций написаны на английском, что облегчает модели сопоставление с английскими вопросами.

Заключение

В ходе выполнения работы были протестированы две модели для семантического поиска по коду: paraphrase-multilingual-mpnet-base-v2 (MPNet) и LaBSE. На основе вычисленной метрики Precision@3, визуального анализа t-SNE и дополнительных исследований лучшей моделью признана LaBSE, так как она показала более высокое значение Precision@3. Это означает, что LaBSE точнее находит релевантные фрагменты кода.

Таким образом, рекомендуется использовать модель LaBSE для задач семантического поиска по коду, так как она обеспечивает более высокую точность и мультиязычность.

Список использованных источников

1. Официальная документация библиотеки sentence-transformers.
URL: <https://www.sbert.net/> (дата обращения: 03.07.2026).
2. Официальная документация библиотеки scikit-learn. URL:
<https://scikit-learn.org/stable/> (дата обращения: 03.07.2026).

Ссылка на репозиторий в github:

<https://github.com/krisfr707/semantic-search-practice/tree/main>